SIMATS ENGINEERING

ASSESSMENT TOOL 1: AI Model Design Exercise

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA01**

COURSE OUTCOME COVERED

CO4: *Develop intelligent software agents using suitable agent architectures and learning techniques.*(BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 1

Annexure A

AI Model Design Exercise

Code of Conduct:

*I (*V Sunil Kumar / 192425292*) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

V Sunil Kumar

Signature of the student:

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Dataset Selection and Data Understanding	10%	
3. Data Preprocessing and Feature Engineering	10%	
4. AI Technique Selection and Justification	10%	
5. Model Design / Architecture	10%	
6. Algorithm / Pseudocode Development	5%	
7. Model Implementation and GitHub Repository	15%	
8. Experimental Design and Results	10%	
9. Performance Evaluation and Metrics	10%	
10. Result Analysis and Interpretation	5%	
11. Limitations and Future Enhancements	5%	
12. Documentation and Technical Presentation	10%	
Total	100%	

Signature of the Faculty Total Marks Awarded

Q6. Non-Linear Classification Using SVM with Kernel Method

1. Problem Statement

The objective of this project is to develop an Artificial Intelligence model for classifying data into two different classes when the classes cannot be separated using a straight-line decision boundary. A conventional linear classifier may fail because the relationship between the input features and class labels is non-linear.

To solve this problem, a Support Vector Machine (SVM) with a Kernel Method is designed. The model transforms the original input space into a higher-dimensional feature space using a suitable kernel function. The Radial Basis Function (RBF) kernel is selected because it can effectively model complex and non-linear decision boundaries.

Input: Numerical features representing the data points.

Output: One of two class labels, Class 0 or Class 1.

AI Task: Binary classification of non-linearly separable data.

2. Objectives

The main objectives of the project are:

1. To understand the problem of non-linear classification.
2. To prepare a suitable dataset for binary classification.
3. To preprocess and split the dataset into training and testing sets.
4. To design an SVM classifier using a kernel function.
5. To select and justify the RBF kernel for non-linear classification.
6. To train the SVM model using the training data.
7. To evaluate the model using suitable performance metrics.
8. To analyze the classification results and identify the strengths and limitations of the model.

3. Dataset Description

For demonstrating non-linear classification, a synthetic two-class dataset can be generated using the make_circles dataset generator available in Scikit-learn.

The dataset contains two classes arranged in a circular pattern. Therefore, a straight line cannot effectively separate the two classes.

Dataset: Synthetic Non-Linear Dataset

Data Generation: make_circles()

Number of Classes: 2

Features: Feature X1 and Feature X2

Target: Class 0 or Class 1

Data Type: Numerical

Classification Type: Binary

Data Pattern: Non-linear / circular

The circular structure makes this dataset suitable for demonstrating the effectiveness of kernel-based SVM

classification.

4. Data Preprocessing

The following preprocessing steps are performed:

Data Generation: A synthetic dataset containing two non-linearly separable classes is generated.

Feature Selection: Two numerical features, X1 and X2, are used as input features.

Data Splitting: The dataset is divided into 80% training data and 20% testing data. The training data is used to learn the classification boundary, while the testing data is used to evaluate the model on unseen samples.

Feature Scaling: Feature scaling is applied using StandardScaler so that the features have comparable scales. Scaling is important for SVM because the distance between data points influences the kernel calculation.

5. Selected AI Technique and Justification

The selected AI technique is Support Vector Machine (SVM) with an RBF kernel.

SVM attempts to find an optimal decision boundary that separates different classes while maximizing the margin between them. However, a linear SVM cannot effectively classify data when the classes have a non-linear structure.

The RBF kernel solves this problem by implicitly mapping the input data into a higher-dimensional feature space.

The RBF kernel is represented as:

$K(x, x') = \exp(-\gamma \, ||x - x'||^2)$

where γ controls the influence of individual training samples. A higher gamma creates a more complex decision boundary, while a lower gamma produces a smoother decision boundary.

The RBF kernel is selected because the dataset is non-linearly separable, it can create curved and complex decision boundaries, it works effectively with numerical features, and it can model non-linear relationships without explicitly creating higher-dimensional features.

6. Model Design / Architecture

The proposed model follows this workflow:

Input Dataset → Data Preprocessing → Feature Scaling → Train-Test Split → SVM with RBF Kernel → Training → Prediction → Performance Evaluation

Parameter Selected Value

Algorithm Support Vector Machine

Kernel RBF

C 1.0

Gamma Scale

Classification Binary

Scaling StandardScaler

Training Split 80%

Testing Split 20%

Model flow:

Input Dataset

↓

Feature Preprocessing

↓

Feature Scaling

↓

Train-Test Split

↓

SVM Classifier with RBF Kernel

↓

Non-Linear Decision Boundary

↓

Class Prediction

↓

Performance Evaluation

The RBF kernel allows the SVM model to construct a curved decision boundary around the data points instead of using a straight line.

7. Algorithm / Pseudocode

Algorithm:

1. Start.
2. Generate or load the non-linear binary classification dataset.
3. Separate the features and target labels.
4. Split the dataset into training and testing sets.
5. Apply feature scaling using StandardScaler.
6. Create an SVM classifier.
7. Select the RBF kernel.
8. Set the parameters C and gamma.
9. Train the SVM model using training data.
10. Predict the class labels for the testing data.
11. Calculate accuracy, precision, recall and F1-score.
12. Generate the confusion matrix.
13. Analyze the classification results.
14. Stop.

Pseudocode:

BEGIN

Load non-linear dataset

Separate features X and target Y

Split X and Y into training and testing data

Scale training and testing features

Create SVM model

Kernel = RBF

C = 1.0

Gamma = scale

Train SVM using training data

Predict classes using testing data

Calculate accuracy, precision, recall and F1-score

Generate confusion matrix

Analyze results

END

8. Implementation

The model can be implemented using Python.

Libraries used include Python, NumPy, Pandas, Scikit-learn, Matplotlib and Seaborn.

Important implementation code:

```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

X, y = make_circles(n_samples=500, noise=0.1, factor=0.5, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = SVC(kernel='rbf', C=1.0, gamma='scale')
```

```
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

9. Experimental Results

The experiment evaluates whether the RBF-based SVM can correctly classify the two non-linearly separable classes.

Experiment 1: SVM with Linear kernel – Poor classification for circular data.

Experiment 2: SVM with RBF kernel – Strong non-linear classification.

Experiment 3: SVM with RBF kernel and scaling – Improved and stable classification.

A confusion matrix can be used to show the number of correctly and incorrectly classified samples.

A decision-boundary visualization can also be included to demonstrate how the RBF kernel separates the two classes.

Actual accuracy, precision, recall and F1-score should be taken from the program output.

10. Performance Evaluation

The following metrics are used to evaluate the model:

Accuracy represents the proportion of correctly classified samples.

Accuracy = Correct Predictions / Total Predictions

Precision measures how many samples predicted as a particular class actually belong to that class.

Precision = TP / (TP + FP)

Recall measures how many actual positive samples are correctly identified.

Recall = TP / (TP + FN)

F1-score provides a balance between precision and recall.

F1 = 2 × (Precision × Recall) / (Precision + Recall)

The confusion matrix provides True Positives, True Negatives, False Positives and False Negatives. These metrics provide a comprehensive evaluation of the SVM classifier.

11. Result Analysis and Interpretation

The SVM with the RBF kernel is appropriate for this non-linear classification problem because the classes cannot be separated using a simple straight-line boundary.

A linear SVM would have difficulty representing the circular relationship between the classes. In contrast, the RBF kernel allows the model to create a flexible curved decision boundary.

The model's performance can be analyzed using the confusion matrix and classification metrics. A high accuracy and F1-score indicate that the model successfully distinguishes between the two classes.

The major strength of the approach is its ability to handle complex non-linear relationships without explicitly transforming the features into a higher-dimensional space.

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Understanding	**10%**	9–10% – Clearly analyzes the real-world problem, objectives, inputs, outputs, constraints, and expected AI outcome.	7–8% – Understands the problem and objectives with minor omissions.	5–6% – Shows partial understanding with some important elements missing.	0–4% – Problem is poorly understood or incorrectly defined.
2. Dataset Selection and Understanding	**10%**	9–10% – Selects a highly relevant dataset and clearly explains features, target, source, size, and characteristics.	7–8% – Appropriate dataset with adequate description and minor omissions.	5–6% – Dataset is usable but description or relevance is limited.	0–4% – Dataset is inappropriate, poorly described, or missing.
3. Data Preprocessing and Feature Engineering	**10%**	9–10% – Performs appropriate cleaning, transformation, encoding, scaling, feature selection, and data splitting with clear justification.	7–8% – Performs most required preprocessing correctly with minor issues.	5–6% – Basic preprocessing is performed but important steps are missing.	0–4% – Preprocessing is inadequate, incorrect, or absent.

4. AI Technique Selection and Justification	**10%**	9–10% – Selects the most appropriate AI technique and provides strong technical justification based on the problem and dataset.	7–8% – Appropriate technique selected with reasonable justification.	5–6% – Technique is acceptable but justification is limited.	0–4% – Inappropriate technique or no meaningful justification.
5. Model Design / Architecture	**10%**	9–10% – Develops a clear, technically sound model architecture with appropriate components, parameters, and workflow.	7–8% – Model is well designed with minor omissions.	5–6% – Basic model design is presented but lacks technical depth.	0–4% – Model design is unclear, incomplete, or inappropriate.
6. Algorithm / Pseudocode	**5%**	5% – Algorithm/pseudocode is complete, logically correct, structured, and clearly represents the model development process.	4% – Mostly correct with minor omissions.	2–3% – Basic algorithm provided but contains gaps.	0–1% – Algorithm is missing or substantially incorrect.
7. Implementation and GitHub Repository	**15%**	13–15% – Fully functional implementation with clean, organized, reproducible code and a complete GitHub repository with README and supporting files.	10–12% – Functional implementation and well-organized repository with minor issues.	7–9% – Partially functional implementation or incomplete repository documentation.	0–6% – Implementation is incomplete/non-functional or GitHub repository is missing.

8. Experimental Design and Results	**10%**	9–10% – Conducts meaningful experiments with appropriate test cases and clearly presents results using tables, graphs, or visualizations.	7–8% – Appropriate experiments with clearly presented results.	5–6% – Limited experiments or basic result presentation.	0–4% – Insufficient, incorrect, or missing experimental results.
9. Performance Evaluation	**10%**	9–10% – Uses appropriate evaluation metrics and provides comprehensive quantitative analysis of model performance.	7–8% – Uses suitable metrics and provides adequate evaluation.	5–6% – Basic evaluation with limited metrics or analysis.	0–4% – Inappropriate metrics or performance evaluation is missing.
10. Result Analysis and Interpretation	**5%**	5% – Provides insightful analysis of results, identifies patterns, errors, strengths, weaknesses, and explains model behavior.	4% – Provides good interpretation with minor gaps.	2–3% – Basic interpretation with limited analysis.	0–1% – Results are not meaningfully analyzed.
11. Limitations and Future Enhancements	**5%**	5% – Clearly identifies realistic limitations and proposes technically relevant and feasible improvements.	4% – Identifies relevant limitations and reasonable improvements.	2–3% – Provides basic limitations and future suggestions.	0–1% – Missing, unrealistic, or poorly explained.
12. Documentation and Technical Presentation	**10%**	9–10% – Report is comprehensive, well organized, technically accurate, professionally presented, and properly referenced.	7–8% – Well documented with minor presentation or referencing issues.	5–6% – Adequate documentation with several gaps.	0–4% – Poorly documented, unclear, or incomplete report.
Total	100%				